Troubleshooting 사례

환경	
클러스터	k3s v1.33.6 (설치를 단순화한 경량 쿠버네티스), 노드 5대 (제어 노드 1, 워커 4)
클라우드	AWS EC2, 서울 리전, VPC(AWS 안의 가상 사설망) `10.180.0.0/20`, 서브넷 2개

1. VPC CNI 사례

1-1. 목적

- 쿠버네티스에서 Pod에 IP를 주고 Pod끼리 통신할 수 있게 연결하는 일은 **CNI**라는 네트워크 플러그인이 맡는다
- 보통의 CNI는 클러스터 안에서만 쓰는 전용 대역(예: `10.42.x.x`)에서 Pod IP를 준다. 클러스터 밖의 VPC는 이 주소를 모른다
- **AWS VPC CNI**는 다르게 동작한다. 노드(EC2)에 보조 네트워크 카드(**ENI**)를 붙이고, 그 카드에 할당된 **VPC의 실제 IP**를 Pod에 하나씩 나눠 준다
- 이 방식으로 Pod가 VPC 안의 다른 자원과 주소 변환이나 터널 없이 직접 통신하게 만드는 것이 목표였다

1-2. 상황

- k3s는 설치하면 **Flannel이라는 CNI가 자동으로 함께 설치**된다. 따로 설정하지 않아도 Pod 네트워크가 바로 동작하게 하기 위해서다
- 다른 CNI를 쓰려면 이 자동 설치를 막아야 한다. 두 CNI가 동시에 Pod 네트워크를 잡으면 충돌하기 때문이다
- 그래서 k3s를 `-flannel-backend none` 옵션으로 시작했다. 이 상태에서는 CNI가 하나도 없으므로, CNI를 직접 설치하기 전까지 노드는 NotReady이고 Pod는 IP를 받지 못한다
- 그 위에 쿠버네티스용 패키지 관리 도구 **Helm**으로 VPC CNI를 설치했다. 설치 중 오류가 세 번 났고, 모두 원인을 찾아 해결했다

오류	무슨 뜻이었나	조치
`conflict occurred while applying object /eniconfigs.crd...`	예전에 kubectl로 직접 설치해 둔 VPC CNI용 리소스 정	기존 정의를 지우고 다시 설치

오류	무슨 뜻이었나	조치
	의(CRD)가 남아 있었다. Helm은 자기가 만들지 않은 리소스를 덮어쓰지 않는다	
`ServiceAccount "aws-node" ... cannot be imported`	같은 이유다. 손으로 만든 실행 계정(ServiceAccount)과 권한 설정이 남아 있었다	관련 리소스를 모두 삭제
`MissingIAMPermissions ... ec2:DescribeNetworkInterfaces`	VPC CNI는 노드의 **IAM 역할**(EC2에 붙인 AWS 권한)로 네트워크 카드를 만들고 IP를 받아 온다. 그 권한이 없었다	모든 노드 역할에 `AmazonEKS_CNI_Policy` 추가

- 그 결과 VPC CNI 에이전트(`aws-node`)는 모든 노드에서 Running, 노드는 모두 Ready, Pod에는 VPC IP가 붙었다. **겉보기에는 완벽했다**

1-3. 증상

- 그런데 Pod에서 시작하는 통신이 전부 실패했다

```
metrics-server  →  dial tcp 10.43.0.1:443: i/o timeout
```

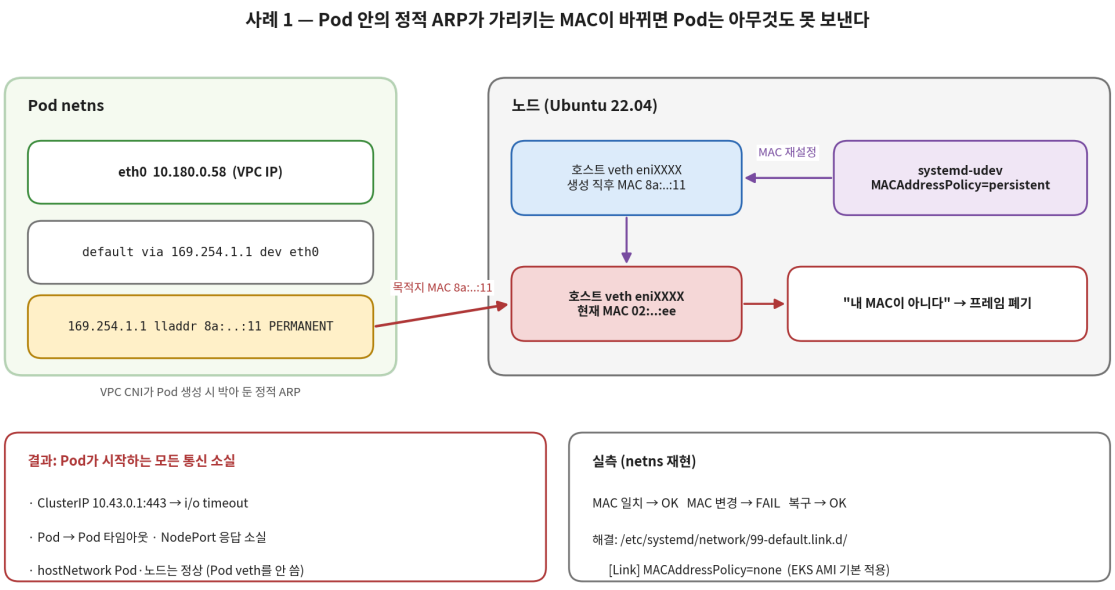
- `metrics-server`는 노드와 Pod의 자원 사용량을 모으는 기본 구성 요소다. 쿠버네티스 API 서버에 접속해야 하는데, API 서버의 **ClusterIP**(Service에 붙는 클러스터 내부용 가상 IP, 여기서는 `10.43.0.1`)로 접속하지 못했다
- 에러는 `i/o timeout`이었다. 상대가 거절한 것이 아니라 **대답 자체가 오지 않았다**는 뜻이다
- Pod에서 다른 노드의 Pod IP로 직접 접속해도 timeout이었다
- **NodePort**(노드 IP의 특정 포트로 들어온 요청을 Pod로 넘겨 주는 방식)로 접속해도 응답이 없었다
- AWS 쪽에서 막는다고 보고 두 가지를 바꿔 봤지만 그대로였다
  - **보안 그룹**(EC2에 붙는 방화벽)을 모두 열었다
  - **출발지·목적지 검사**(EC2가 자기 IP가 아닌 패킷을 버리는 기능)를 껐다
- 결국 CNI를 Cilium(다른 CNI)으로 교체했고, 그러자 모든 통신이 정상이 됐다

1-4. 당시 판단

- 당시 결론은 “VPC CNI는 AWS의 관리형 쿠버네티스(**EKS**) 전용이라 k3s와 근본적으로 맞지 않는다”였다
- 다시 확인해 보니 맞지 않는 판단이었다
  - VPC CNI는 kops(AWS에 쿠버네티스를 직접 구축하는 도구)처럼 EKS가 아닌 클러스터에서도 공식 옵션으로 쓰인다
  - “VPC CNI의 패킷은 노드의 방화벽 규칙(iptables)을 거치지 않는다”고 봤지만, Pod 패킷은 Pod와 노드를 잇는 가상 케이블(**veth**)을 통해 노드로 들어온 뒤 밖으로 나간다
  - “Pod 간 통신 실패는 kube-proxy 탓”이라고 봤지만, **kube-proxy**는 ClusterIP를 실제 Pod IP로 바꿔 주는 구성 요소다. Pod IP로 직접 가는 통신에는 관여하지 않는다

1-5. 실제 원인

정적 ARP 불일치



- 증상이 같은 공개 이슈가 있다: **“Ubuntu 22.04에서 Pod가 ClusterIP와 통신하지 못함”** (aws/amazon-vpc-cni-k8s#2103)
- 이해에 필요한 배경 두 가지
  - **MAC 주소**: 네트워크 카드마다 붙은 고유 번호. 같은 망 안에서 패킷을 전달할 때는 IP가 아니라 이 번호로 받는 쪽을 찾는다

    - **ARP**: "이 IP의 MAC은 무엇인가"를 물어서 기억해 두는 장치. 보통은 필요할 때 물어보지만, 고정으로 적어 둘 수도 있다. 이것을 **정적 ARP**라고 한다

- 일어난 일
    1. VPC CNI는 Pod를 만들 때, Pod 안에 "게이트웨이(`169.254.1.1`)의 MAC은 이것"이라고 **고정으로** 적어 둔다. 게이트웨이는 Pod가 밖으로 보내는 모든 패킷을 일단 넘기는 곳이고, 적어 둔 MAC은 노드 쪽 가상 케이블 끝(호스트 veth)의 MAC이다
    2. Ubuntu 22.04의 **systemd**(리눅스에서 장치와 서비스를 관리하는 시스템 프로그램)는 새로 생긴 네트워크 장치의 MAC을 자기 규칙(`MACAddressPolicy=persistent`)으로 다시 정한다
    3. Pod는 예전 MAC 앞으로 패킷을 보내고, 노드 쪽 veth는 자기 MAC이 아니라며 버린다
    4. 그래서 **Pod가 시작하는 통신은 전부 사라진다**. ClusterIP, Pod 간 통신, NodePort의 응답이 한꺼번에 실패한 이유다
- 보안 그룹과 출발지·목적지 검사가 소용없었던 것도 같은 이유다. 패킷이 노드 밖으로 나가기 전에, 노드 안에서 버려졌다
- Pod를 흉내 낸 독립 네트워크 공간(netns)으로 재현한 결과

```
OK   Pod → 노드   (MAC 일치)
FAIL Pod → 노드   (노드 쪽 veth MAC만 변경)
OK   Pod → 노드   (MAC 복구)
```

1-6. 해결

- 확인 방법: Pod가 기억하는 게이트웨이 MAC과 노드 veth의 실제 MAC을 비교한다. 다르면 이 문제다

```
ip netns exec <pod-ns> ip neigh show 169.254.1.1    # Pod가 기억하는 게이트웨이 MAC
ip link show <eniXXXX>                               # 노드 veth의 실제 MAC
```

- 해결 방법: systemd가 MAC을 바꾸지 않도록 설정 파일을 하나 추가하고 Pod를 다시 만든다. AWS가 EKS용 노드 이미지에 기본으로 넣어 두는 설정이다

```
# /etc/systemd/network/99-default.link.d/aws-cni-workaround.conf
[Link]
MACAddressPolicy=none
```

1-7. 배운 점

- **CNI 에이전트가 Running이고 Pod에 IP가 붙었다고 해서 패킷이 흐르는 것은 아니다**
- CNI는 노드 OS 설정에 기대는 부분이 있다. 같은 CNI라도 노드 OS 이미지에 따라 결과가 달라진다
- CNI를 교체해도 이미 떠 있던 Pod는 예전 CNI가 준 IP를 그대로 가진다. 당시 CoreDNS(클러스터 내부 DNS 서버)만 예전 대역에 남아 있었다. 교체 후에는 모든 Pod를 다시 만들어야 한다

---

2. IMDS 사례

2-1. 목적

- **ALB**(AWS가 제공하는 HTTP 로드밸런서)를 쿠버네티스 설정만으로 자동 생성·관리하고 싶었다
- **AWS Load Balancer Controller**를 클러스터에 설치하면, 쿠버네티스에 Ingress(외부 HTTP 요청을 어느 서비스로 보낼지 적는 리소스)를 만들 때 컨트롤러가 그에 맞는 ALB를 대신 만들어 준다
- 컨트롤러가 AWS API를 호출하려면 AWS 자격 증명이 필요하다. 여기서는 노드에 붙인 IAM 역할의 자격 증명을 빌려 쓰는 방식이었다
- EC2 안의 프로그램이 그 자격 증명을 받는 통로가 **IMDS**(인스턴스 메타데이터 서비스)다. 모든 EC2 안에서 `169.254.169.254` 주소로 접근한다

2-2. 상황

- 첫 사례 이후 CNI는 **Cilium**으로 바뀌어 있었다. 다른 노드의 Pod로 가는 패킷은 **VXLAN 터널**(Pod 패킷을 노드 주소가 적힌 패킷으로 한 번 더 감싸 보내는 방식)로 보냈다
- IMDS는 보안 권장 설정을 쓰고 있었다
  - **IMDSv2만 허용**: 먼저 토큰을 받아야 메타데이터를 읽을 수 있는 방식
  - **hop limit 2**: 토큰 응답이 몇 단계까지 넘겨질 수 있는지 정하는 값 (자세한 뜻은 2-5)

- 워커 노드의 IAM 역할에는 ALB 제어 권한이 이미 들어 있었다

2-3. 증상

- 컨트롤러가 ALB를 조회하는 단계에서 자격 증명을 얻지 못했다

```
get credentials: failed to refresh cached credentials, no EC2 IMDS role found,
operation error ec2imds: GetMetadata, canceled, context deadline exceeded
```

- "IMDS에서 역할을 찾지 못했고, 시간이 초과됐다"는 내용이다. 권한이 없다는 에러가 아니라 **IMDS에서 대답이 오지 않은** 에러였다

2-4. 당시 판단

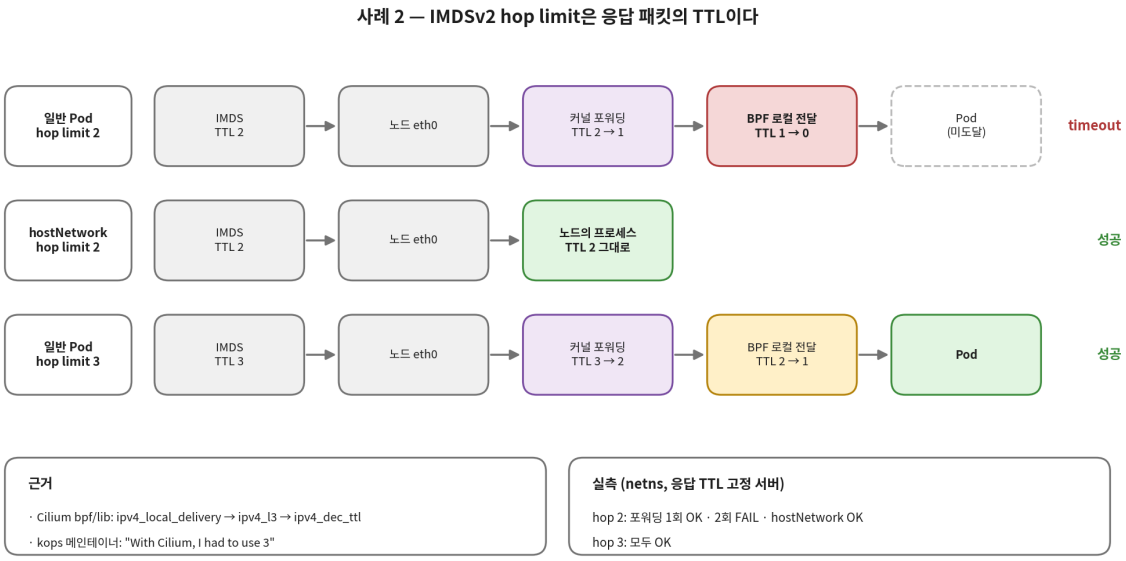
- 원인을 좁혀 간 과정은 정확했다

확인한 것	결과	이로써 알게 된 것
EC2 메타데이터 설정	IMDSv2, hop limit 2	IMDS는 켜져 있다
노드 IAM 역할과 정책	ALB 정책 포함	권한은 있다
노드에서 직접 IMDS 요청	성공	IMDS 자체는 정상이다
노드에서 AWS CLI로 ALB 조회	성공	IAM 권한도 실제로 동작한다
일반 Pod에서 IMDS 요청	**timeout**	Pod에서만 안 된다
hostNetwork Pod(자기 네트워크 공간 없이 노드의 네트워크를 그대로 쓰는 Pod)에서 IMDS 요청	**성공**	노드 네트워크를 쓰면 된다

- 여기까지로 "권한 문제가 아니라 **Pod에서 IMDS까지 가는 길의 문제**"라는 결론은 맞았다
- 다만 그 길의 문제를 "Cilium 터널 모드에서는 link-local 주소(`169.254.x.x`처럼 같은 장비 안에서만 쓰는 특수 주소)가 터널로 넘어가지 않는다"고 봤는데, 이 부분이 틀렸다
  - 터널은 다른 노드의 Pod로 갈 때만 쓴다. IMDS는 다른 노드가 아니라 내 EC2 바로 앞에 있으니 처음부터 터널로 갈 트래픽이 아니다
  - 대안으로 적어 둔 iptables 마스커레이드 규칙(Pod에서 나가는 패킷의 출발지 주소를 노드 IP로 바꾸는 규칙)도, Cilium이 이미 같은 일을 하고 있었다 (Cilium 소스로 확인)

2-5. 실제 원인

IMDS 응답 TTL



- 원인은 설정값 하나, **hop limit 2**였다
- 패킷에는 **TTL**이라는 수명 값이 있다. 패킷이 한 장치에서 다음 장치로 넘겨질 때마다 1씩 줄고, 0이 되면 버려진다
- IMDSv2의 hop limit은 토큰 응답 패킷에 붙이는 이 TTL이다. 노드 안의 프로그램은 받을 수 있지만, 노드를 거쳐 더 멀리 있는 곳은 받지 못하게 하려는 보안 장치다
- Cilium 환경에서는 응답이 Pod까지 가는 동안 TTL이 **두 번** 줄었다
  1. 노드의 리눅스 커널이 응답을 Cilium 쪽으로 넘기면서 1 감소 (2 → 1)
  2. Cilium의 **eBPF 프로그램**(커널 안에서 패킷을 처리하는 Cilium의 작은 프로그램)이 응답을 Pod에 넘기면서 1 감소 (1 → 0) → **Pod에 닿기 전에 폐기**
- hostNetwork Pod는 노드에서 바로 받으니 넘겨지는 단계가 없어 TTL이 그대로였고, 그래서 성공했다. 관찰된 패턴과 정확히 일치한다
- 근거
  - Cilium 소스: Pod로 전달하는 함수(`ipv4_local_delivery`)가 TTL을 줄인다 (`ipv4_dec_ttl`)
  - kops 메인테이너의 기록: “Cilium에서는 hop limit을 3으로 해야 동작했다”
- 응답 TTL을 고정한 가짜 IMDS 서버로 재현한 결과

```
OK   hop limit 2 | 응답이 1번 넘겨짐
FAIL hop limit 2 | 응답이 2번 넘겨짐
OK   hop limit 2 | hostNetwork (넘겨짐 없음)
OK   hop limit 3 | 응답이 2번 넘겨짐
```

2-6. 해결

- 해결 방법은 세 가지이고, **보안 범위가 다르다**

방법	효과	주의할 점
hop limit을 3으로 올림	모든 Pod가 IMDS를 쓸 수 있다	모든 Pod가 노드의 IAM 역할을 가져갈 수 있다. 이번 노드 역할에는 ALB·DynamoDB 권한이 있었다
컨트롤러만 hostNetwork	컨트롤러 Pod만 IMDS를 쓴다	노드 네트워크를 공유한다. 클러스터 내부 DNS를 계속 쓰려면 `dnsPolicy: ClusterFirstWithHostNet` 설정이 필요하다
IRSA (Pod의 실행 계정마다 별도 IAM 역할을 연결하는 방식)	각 Pod가 필요한 권한만 받는다	k3s에서는 AWS와의 인증 연동(OIDC)과 보조 구성 요소(웹훅)를 직접 설치해야 한다. 대신 hop limit은 1로 잠글 수 있다

- 당시 선택한 hostNetwork는 범위가 좁아서 해커톤에서는 합리적이었다. 운영 환경이라면 IRSA가 맞다

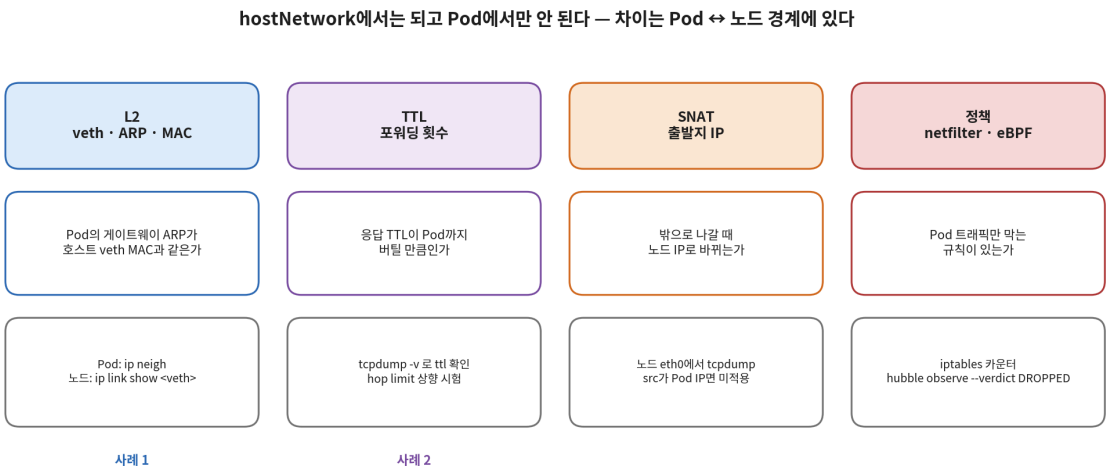
2-7. 배운 점

- **범위를 좁히는 것과 원인을 확정하는 것은 다르다**. 범위는 정확히 좁혔지만, 마지막 원인은 그럴듯한 설명으로 채웠다
- 원인은 실제 값으로 확인해야 한다. 노드에서 패킷 캡처 도구(`tcpdump -v`)로 응답의 `ttl 2`를 봤다면 바로 확정할 수 있었다
- 같은 hop limit이라도 CNI가 패킷을 몇 번 넘기는지에 따라 결과가 달라진다

---

3. 공통점

Pod-노드 경계



- 두 사례 모두 **hostNetwork에서는 되고 일반 Pod에서만 안 됐다**
- hostNetwork Pod는 Pod와 노드 사이의 경계(가상 케이블, 게이트웨이, 중간에 넘겨지는 단계)를 통째로 건너뛴다. 그러니 이 패턴이 보이면 원인은 그 경계에 있다
- 경계에서 확인할 것

확인할 것	질문	이번 사례
게이트웨이 MAC	Pod가 기억하는 게이트웨이 MAC과 노드 veth의 MAC이 같은가	VPC CNI 사례
TTL	응답이 Pod까지 버틸 만큼 수명이 남는가	IMDS 사례
출발지 주소 변환	밖으로 나갈 때 출발지가 노드 IP로 바뀌는가	—
차단 규칙	Pod 트래픽만 막는 방화벽·정책 규칙이 있는가	—

- **"hostNetwork로 바꾸니 된다"는 원인이 경계에 있다는 증거일 뿐, 원인 자체는 아니다**. 두 사례 모두 이 우회로 넘어가면서 원인을 다르게 적었다